Modul Praktikum 9

UNIT TESTING DENGAN JUNIT 5

D3 Teknik Informatika

POLITEKNIK NEGERI BANDUNG

Daftar Isi

Daftar Isi..... 2
1. Modul Overview..... 2
2. Persiapan Praktikum..... 3
3. Praktikum I - Test Pertama dan AAA..... 4
4. Praktikum II - Exception dan Integritas State..... 6
5. Praktikum III - Test Fixture dan Parameterized Test..... 7
5. Praktikum III - Test Fixture dan Parameterized Test (lanjutan)..... 8
6. Praktikum IV - Testing Inheritance dan Polymorphism..... 9
7. Praktikum V - Mini TDD..... 10
8. Sintesis Hasil Praktikum..... 11
9. Laporan dan Pengumpulan..... 12
10. Checkpoint dan Refleksi..... 12

1. Modul Overview

Pada pertemuan sebelumnya, kita biasanya mengecek program dengan menjalankannya lalu melihat output. Pada praktikum ini, kita akan memakai JUnit agar pengecekan dapat dilakukan otomatis dan diulang kapan saja. Fokusnya bukan menghafal anotasi, tetapi memastikan perilaku class sesuai yang diharapkan.

1.1 Tujuan Pembelajaran

- Menjelaskan perbedaan antara program yang sekadar berjalan dan perilaku yang sudah diuji.
- Menyiapkan project Maven dan menjalankan JUnit 5 (Jupiter).
- Menulis unit test dengan pola Arrange - Act - Assert (AAA).
- Menggunakan assertEquals, assertAll, assertThrows, dan assertDoesNotThrow secara tepat.
- Menguji exception sekaligus memastikan kondisi object tetap benar ketika operasi gagal.
- Mengurangi duplikasi setup test menggunakan @BeforeEach.
- Menggunakan @ParameterizedTest untuk menguji beberapa data pada perilaku yang sama.
- Menguji inheritance dan polymorphism melalui reference superclass.
- Menerapkan siklus TDD sederhana: Red - Green - Refactor.

1.2 Cara Kerja di Setiap Langkah

PREDICT -> CODE -> RUN -> OBSERVE -> EXPLAIN

Sebelum menjalankan test, prediksi dulu hasilnya: hijau atau merah? Setelah itu jalankan, amati hasilnya, lalu jelaskan penyebabnya. Jika test gagal, jangan langsung mengubah expected. Cek dulu apakah masalah ada pada test, kode, atau asumsi Anda.

1.3 Batas Praktikum

- Fokus wajib: @Test, AAA, assertion dasar, assertThrows, @BeforeEach, parameterized test, inheritance/polymorphism test, dan mini TDD.
- @BeforeAll, @AfterAll, @Nested, dan anotasi lanjutan diposisikan sebagai challenge opsional bila waktu cukup.
- Test memeriksa perilaku publik object, bukan mengakses field private secara langsung.
- Setiap test harus berdiri sendiri dan tidak bergantung pada test lain.

1.4 Penggunaan AI

Coba cari penyebab error sendiri terlebih dahulu. AI boleh digunakan setelah Anda memiliki hasil compile/run untuk meminta petunjuk atau penjelasan. Jangan meminta AI membuat seluruh test. Setiap saran tetap harus diperiksa dengan menjalankan test kembali.

2. Persiapan Praktikum

2.1 Struktur Project

```
praktikum9/
├── pom.xml
└── src/
    ├── main/java/id/ac/polban/pbo/
    │   ├── BankAccount.java
    │   ├── InsufficientBalanceException.java
    │   └── SavingsAccount.java
    └── test/java/id/ac/polban/pbo/
        ├── BankAccountTest.java
        └── SavingsAccountTest.java
```

2.2 Konfigurasi Maven

Gunakan JDK 17+ dan Maven. Tambahkan JUnit 5 sebagai dependency khusus untuk testing dengan scope test. Simpan class test di src/test/java dan gunakan akhiran Test.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.junit</groupId>
      <artifactId>junit-bom</artifactId>
      <version>5.11.4</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

2.3 Verifikasi Environment

- Jalankan java -version.
- Jalankan mvn -v.
- Jalankan mvn test pada project kosong untuk memastikan Maven dan JUnit dikenali.

Blok	Fokus	Waktu
Praktikum I	Test pertama dan AAA	40 menit
Praktikum II	Exception dan integritas state	40 menit
Praktikum III	Fixture dan parameterized test	35 menit
Praktikum IV	Inheritance dan polymorphism test	35 menit
Praktikum V	Mini TDD	30 menit

Sintesis + buffer	Analisis, debugging, pengumpulan	20 menit

3. Praktikum I - Test Pertama dan AAA

Kita mulai dari class BankAccount dan method deposit. Kasusnya sengaja sederhana agar fokus pada cara menulis unit test.

Step 1 - Bangun BankAccount baseline (10 menit)

```
package id.ac.polban.pbo;

public class BankAccount {
    private final String accountNumber;
    private final String owner;
    protected double balance;

    public BankAccount(String accountNumber, String owner,
                       double initialBalance) {
        if (accountNumber == null || accountNumber.isBlank())
            throw new IllegalArgumentException(
                "Nomor rekening wajib diisi");
        if (initialBalance < 0)
            throw new IllegalArgumentException(
                "Saldo awal tidak boleh negatif");
        this.accountNumber = accountNumber;
        this.owner = owner;
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0)
            throw new IllegalArgumentException(
                "Jumlah setoran harus lebih dari 0");
        balance += amount;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getOwner() { return owner; }
    public double getBalance() { return balance; }
}
```

Checkpoint 1

Jika Anda hanya menjalankan main() satu kali dan melihat saldo akhir, kondisi apa saja yang belum terbukti? Tuliskan minimal dua.

Step 2 - Tulis test pertama (15 menit)

```
@Test
void deposit_menambahSaldo() {
    // Arrange
    BankAccount acc =
        new BankAccount("001", "Andi", 100_000);

    // Act
    acc.deposit(50_000);
```

```
    // Assert
    assertEquals(150_000, acc.getBalance(), 0.001);
}
```

- Prediksi hasil sebelum run.
- Jalankan mvn test.
- Ubah expected 150_000 menjadi 160_000 dan jalankan lagi.
- Simpan pesan kegagalan, lalu kembalikan expected ke nilai yang benar.

Checkpoint 2

Mengapa sengaja membuat expected salah justru membantu memahami fungsi unit test? Apa informasi yang diberikan expected dan actual?

Step 3 - Beberapa assertion dalam satu test (15 menit)

```
@Test
void constructor_menyimpanDataDenganBenar() {
    BankAccount acc =
        new BankAccount("001", "Andi", 100_000);

    assertAll("data rekening",
        () -> assertEquals("001", acc.getAccountNumber()),
        () -> assertEquals("Andi", acc.getOwner()),
        () -> assertEquals(100_000, acc.getBalance(), 0.001)
    );
}
```

4. Praktikum II - Exception dan Integritas State

Exception juga perlu diuji. Kita tidak hanya memastikan exception muncul, tetapi juga memastikan object tetap berada pada kondisi yang benar setelah operasi gagal.

Step 4 - Tambahkan checked exception dan withdraw (15 menit)

```
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

public void withdraw(double amount)
        throws InsufficientBalanceException {
    if (amount <= 0)
        throw new IllegalArgumentException(
            "Jumlah penarikan harus lebih dari 0");
    if (amount > balance)
        throw new InsufficientBalanceException(
            "Saldo tidak mencukupi");
    balance -= amount;
}
```

Step 5 - Test exception dan state (15 menit)

```
@Test
void withdraw_melebihiSaldo_ditolak() {
    BankAccount acc =
        new BankAccount("001", "Andi", 100_000);

    assertThrows(
        InsufficientBalanceException.class,
        () -> acc.withdraw(200_000)
    );

    assertEquals(100_000, acc.getBalance(), 0.001);
}
```

Checkpoint 3

Mengapa saldo tetap 100.000 juga perlu diperiksa? Bug apa yang mungkin lolos jika Anda hanya mengecek exception?

Step 6 - Happy path dan pesan exception (10 menit)

```
@Test
void withdraw_valid_tidakMelempar() {
    BankAccount acc =
        new BankAccount("001", "Andi", 100_000);

    assertDoesNotThrow(() -> acc.withdraw(30_000));
    assertEquals(70_000, acc.getBalance(), 0.001);
}

@Test
void constructor_saldoNegatif_ditolak() {
    IllegalArgumentException ex = assertThrows(
        IllegalArgumentException.class,
        () -> new BankAccount("001", "Andi", -1));

    assertEquals("Saldo awal tidak boleh negatif",
                 ex.getMessage());
}
```

Saat menguji exception, jangan hanya memastikan exception muncul. Periksa juga bahwa object tetap berada pada kondisi yang benar setelah operasi gagal.

5. Praktikum III - Test Fixture dan Parameterized Test

Step 7 - Temukan duplikasi setup (8 menit)

Beberapa test membuat BankAccount dengan data yang sama. Sebelum mengubah code, tandai bagian setup yang berulang.

Step 8 - Gunakan @BeforeEach (12 menit)

```
class BankAccountTest {
    private BankAccount sender;
    private BankAccount receiver;
```

```
    @BeforeEach
    void setUp() {
        sender = new BankAccount("001", "Andi", 100_000);
        receiver = new BankAccount("002", "Budi", 0);
    }

    // test berikutnya menggunakan object baru
    // untuk setiap method test
}
```

Checkpoint 4

Mengapa @BeforeEach membuat object baru untuk setiap test? Hubungkan dengan prinsip Independent.

Step 9 - Parameterized test untuk data invalid (15 menit)

```
@ParameterizedTest(name = "setoran {0} ditolak")
@ValueSource(doubles = {0, -1, -50_000})
void deposit_tidakValid(double amount) {
    assertThrows(
        IllegalArgumentException.class,
        () -> sender.deposit(amount)
    );
}
```

Step 10 - Tambahkan boundary value (10 menit)

- Pilih satu nilai boundary yang relevan untuk deposit atau withdraw.
- Tambahkan ke parameterized test.
- Jelaskan mengapa nilai tersebut lebih bernilai dibanding menambah nilai acak.

5. Praktikum III - Test Fixture dan Parameterized Test (lanjutan)

Step 11 - Beberapa input dan expected output dengan @CsvSource (20 menit)

```
@ParameterizedTest(name = "saldo {0} - tarik {1} = {2}")
@CsvSource({
    "100000, 10000,   90000",
    "100000, 100000,  0",
    "50000,  25000.5, 24999.5"
})
void withdraw_berbagaiNilai(
        double awal, double tarik, double sisa)
        throws InsufficientBalanceException {

    BankAccount acc =
        new BankAccount("009", "Dewi", awal);

    acc.withdraw(tarik);

    assertEquals(sisa, acc.getBalance(), 0.001);
}
```

Checkpoint 5

Kapan parameterized test lebih tepat dibanding tiga method @Test yang hampir sama? Kapan justru test terpisah lebih mudah dibaca?

Step 12 - Challenge opsional: lifecycle dan grouping

- Tambahkan @DisplayName pada class dan beberapa test.
- Eksperimen @BeforeAll dan @AfterAll; catat urutan output.
- Kelompokkan test transfer dengan @Nested jika waktu cukup.

Catatan

Challenge ini tidak wajib. Utamakan pemahaman test inti sebelum mencoba anotasi tambahan.

Prinsip Test yang Baik

Prinsip	Makna praktis
Fast	Cepat dijalankan berulang.
Independent	Tidak bergantung urutan atau state test lain.
Repeatable	Hasil konsisten untuk kondisi yang sama.
Self-validating	PASS/FAIL ditentukan otomatis.
Timely	Ditulis dekat dengan saat behavior dibuat/diubah.

6. Praktikum IV - Testing Inheritance dan Polymorphism

Unit test juga dapat membuktikan polymorphism: meskipun reference bertipe superclass, aturan subclass tetap dijalankan sesuai object yang sebenarnya.

Step 13 - Buat SavingsAccount (15 menit)

```
public class SavingsAccount extends BankAccount {
    public static final double MIN_BALANCE = 50_000;
    private final double interestRate;

    public SavingsAccount(String accountNumber, String owner,
                          double initialBalance,
                          double interestRate) {
        super(accountNumber, owner, initialBalance);
        if (interestRate < 0)
            throw new IllegalArgumentException(
                "Bunga tidak boleh negatif");
        this.interestRate = interestRate;
    }
```

```
    @Override
    public void withdraw(double amount)
            throws InsufficientBalanceException {
        if (getBalance() - amount < MIN_BALANCE)
            throw new InsufficientBalanceException(
                "Saldo minimum Rp50.000 harus dipertahankan");
        super.withdraw(amount);
    }

    public void addInterest() {
        balance += balance * interestRate;
    }
}
```

Step 14 - Test melalui reference superclass (15 menit)

```
@Test
void aturanSubclass_berlakuLewatReferenceInduk() {
    BankAccount acc = new SavingsAccount(
        "003", "Citra", 100_000, 0.05);

    assertThrows(
        InsufficientBalanceException.class,
        () -> acc.withdraw(60_000)
    );

    assertEquals(100_000, acc.getBalance(), 0.001);
}
```

Checkpoint 6

Variabel acc bertipe BankAccount, tetapi aturan SavingsAccount tetap berjalan. Konsep OOP apa yang terlihat di sini? Apa yang menentukan method override yang dijalankan?

Step 15 - Test boundary saldo minimum (5 menit)

```
@Test
void withdraw_tepatBatasMinimum_diterima()
        throws InsufficientBalanceException {
    SavingsAccount savings = new SavingsAccount(
        "003", "Citra", 100_000, 0.05);

    savings.withdraw(50_000);

    assertEquals(SavingsAccount.MIN_BALANCE,
                 savings.getBalance(), 0.001);
}
```

7. Praktikum V - Mini TDD

Pada bagian ini kita mencoba TDD sederhana. Kali ini test ditulis lebih dulu untuk perilaku yang belum didukung oleh kode.

Step 16 - Baseline transfer (5 menit)

```
public void transfer(BankAccount target, double amount)
```

```
        throws InsufficientBalanceException {
    withdraw(amount);
    target.deposit(amount);
}
```

Step 17 - RED: tulis test lebih dahulu (10 menit)

Aturan baru: transfer dengan target null harus ditolak dengan IllegalArgumentException dan saldo pengirim tidak boleh berkurang.

```
@Test
void transfer_targetNull_ditolak() {
    BankAccount sender =
        new BankAccount("001", "Andi", 100_000);

    assertThrows(
        IllegalArgumentException.class,
        () -> sender.transfer(null, 20_000)
    );

    assertEquals(100_000, sender.getBalance(), 0.001);
}
```

- Jalankan test sebelum mengubah BankAccount.transfer().
- Simpan bukti test gagal (RED).
- Jelaskan mengapa kegagalan tersebut memang diharapkan.

Step 18 - GREEN: perbaikan minimal (10 menit)

Ubah transfer() seperlunya agar aturan baru terpenuhi dan test menjadi hijau. Jangan mengubah expected hanya agar test lulus.

Step 19 - REFACTOR: rapikan tanpa mengubah behavior (5 menit)

Jika ada duplikasi atau nama yang kurang jelas, rapikan setelah seluruh test hijau. Jalankan kembali semua test untuk memastikan perilakunya tetap sama.

Checkpoint 7

Mengapa pada TDD kita perlu melihat test gagal terlebih dahulu? Apa artinya ketika test yang sama kemudian berubah hijau?

Fokus bagian ini adalah mencoba langsung urutan Red -> Green -> Refactor. Anda belum dituntut menggunakan TDD untuk seluruh aplikasi.

8. Sintesis Hasil Praktikum

Isi tabel berdasarkan test yang benar-benar Anda jalankan. Gunakan hasil praktik, bukan hafalan.

Kasus	Behavior yang dibuktikan	Assertion / mekanisme
Deposit	Saldo bertambah sesuai amount	assertEquals
Withdraw invalid	Exception muncul dan saldo tetap	assertThrows + assertEquals
Parameterized	Satu behavior diuji dengan beberapa data	@ValueSource / @CsvSource
SavingsAccount	Override tetap berlaku via reference BankAccount	assertThrows / assertEquals
Mini TDD	Test gagal sebelum perbaikan dan lulus sesudahnya	Red -> Green -> Refactor

Step 20 - Refleksi singkat (15 menit)

- Pilih satu test yang menurut Anda paling penting dan jelaskan bug apa yang dapat dideteksinya.
- Bandingkan mengecek output main() secara manual dengan menjalankan unit test.
- Jelaskan mengapa test exception sebaiknya juga memeriksa kondisi object setelah operasi gagal.
- Jelaskan mengapa @BeforeEach membantu membuat test tetap independen, tetapi tidak semua bagian Arrange harus dipindahkan ke sana.
- Jelaskan bagaimana test SavingsAccount membuktikan polymorphism.

Challenge Mandiri

- Tentukan aturan transfer ke rekening sendiri: ditolak atau diabaikan. Tulis test lebih dulu.
- Tulis test bunga negatif pada SavingsAccount dan periksa pesan exception.
- Jika waktu cukup, tambahkan Shape/Circle/Rectangle dan uji behavior area() melalui reference Shape.

9. Laporan dan Pengumpulan

9.1 Isi Laporan PDF

- Identitas: nama, NIM, kelas.
- Bukti first test hijau dan satu bukti failure saat expected sengaja dibuat salah.
- Test exception yang juga membuktikan state object tetap.
- Bukti @BeforeEach dan parameterized test.
- Test SavingsAccount melalui reference BankAccount.
- Bukti mini TDD: RED dan GREEN.
- Jawaban checkpoint 1-7 dan refleksi pada Bagian 10.4.

9.2 Format Bukti

- Pastikan potongan code terbaca. Hindari screenshot seluruh IDE jika teks menjadi terlalu kecil.
- Beri caption singkat pada hasil test: apa yang diuji dan apa kesimpulannya.
- Simpan bukti test yang gagal sebelum diperbaiki.

- Pastikan source final sama dengan yang dijelaskan pada laporan.

9.3 Struktur Folder

```
NIM_Nama_PBO9/
├── Laporan_PBO9.pdf
└── source/
    ├── pom.xml
    └── src/
        ├── main/java/...
        └── test/java/...
```

10. Checkpoint dan Refleksi

10.1 Definition of Done

1. Project Maven dapat menjalankan mvn test tanpa error konfigurasi.
2. Saya memiliki test constructor/deposit dan dapat menjelaskan pola AAA.
3. Saya memiliki test withdraw valid dan invalid.
4. Test exception juga memeriksa state object setelah failure.
5. @BeforeEach digunakan untuk setup yang memang berulang.
6. Minimal satu @ParameterizedTest berhasil dijalankan.
7. Test SavingsAccount membuktikan behavior subclass melalui reference BankAccount.
8. Mini TDD menunjukkan bukti RED sebelum perbaikan dan GREEN setelah perbaikan.
9. Saya dapat menjelaskan mengapa setiap test penting, bukan hanya menunjukkan semua test hijau.

10.2 Skenario Uji

ID	Skenario	Expected Result
T9-01	deposit 50.000 pada saldo 100.000	saldo 150.000
T9-02	expected sengaja salah	test merah dengan expected/actual
T9-03	withdraw melebihi saldo	exception; saldo tidak berubah
T9-04	withdraw valid	tidak melempar; saldo berkurang
T9-05	deposit 0/negatif	IllegalArgumentException
T9-06	SavingsAccount via BankAccount	override subclass tetap berlaku
T9-07	transfer target null sebelum perbaikan	RED
T9-08	transfer target null sesudah perbaikan	GREEN; saldo tetap

10.3 Self Assessment

Pernyataan	Skor 1-5
Saya dapat menjelaskan Arrange, Act, dan Assert dari test saya sendiri.	
Saya dapat membedakan failure karena test salah dengan failure karena kode produksi salah.	
Saya dapat menggunakan assertThrows untuk memeriksa kontrak exception.	
Saya memahami mengapa state object perlu diperiksa setelah operasi gagal.	
Saya dapat menggunakan @BeforeEach tanpa membuat test saling bergantung.	
Saya dapat menggunakan parameterized test untuk data yang memiliki behavior sama.	
Saya dapat menjelaskan mengapa reference BankAccount tetap menjalankan override SavingsAccount.	
Saya memahami urutan Red - Green - Refactor pada mini TDD.	

10.4 Refleksi - Jawab dari pengalaman praktikum

1. Apa perbedaan antara kode yang berhasil dijalankan dan behavior yang teruji?

2. Mengapa unit test sebaiknya menguji behavior publik object, bukan field private?

3. Pada withdraw gagal, apa hubungan exception dengan encapsulation dan integritas state?

4. Mengapa satu object bersama untuk seluruh test dapat membuat hasil test tidak independen?

5. Kapan parameterized test membantu, dan kapan test terpisah lebih mudah dibaca?

6. Apa yang dibuktikan ketika variabel BankAccount menjalankan aturan withdraw SavingsAccount?

7. Apa manfaat melihat test gagal pada fase RED sebelum menulis implementasi?

8. Lengkapi dengan bahasa Anda sendiri: "Unit test bukan untuk membuktikan program tidak pernah salah, tetapi ..."